PrincipiosSOLID

Aplicacion del Principio de Unica Responsabilidad (SRP)

Proyecto Java (Maven) de ejemplo academico que refactoriza una clase Empleado monolitica para aplicar el Principio de Unica Responsabilidad (SRP), primero de los principios SOLID.

Elemento	**Valor**
Grupo / Artefacto	com.usil : PrincipiosSOLID : 1.0
Empaquetado	jar
Version de Java	21 (maven.compiler.source / target)
Clase principal	com.usil.PrincipiosSOLID

1. Estructura del proyecto

```
PrincipiosSOLID/
+-- pom.xml
+-- README.md
+-- src/
    +-- main/
        +-- java/
            +-- com/
                +-- usil/
                    +-- PrincipiosSOLID.java         <- clase main (punto de entrada)
                    +-- model/
                    |   +-- Empleado.java            <- clase abstracta base
                    |   +-- Gerente.java
                    |   +-- Desarrollador.java
                    |   +-- Practicante.java
                    +-- service/
                    |   +-- EmpleadoService.java     <- regla de negocio
                    +-- repository/
                    |   +-- EmpleadoRepository.java  <- persistencia
                    +-- report/
                        +-- EmpleadoReport.java      <- reporteria
```

2. Paquetes y responsabilidades

2.1 com.usil - Capa de arranque

Contiene unicamente el punto de entrada de la aplicacion.

Clase	**Responsabilidad**
`PrincipiosSOLID`	Clase main. Crea las instancias de empleados, invoca a EmpleadoService para calcular los pagos e imprime los resultados en consola. Es el orquestador: no

	contiene reglas de calculo, solo coordina.

2.2 com.usil.model - Capa de dominio

Representa que es un empleado y como se calcula su propio pago. No sabe nada de bases de datos ni de reportes.

Clase	Responsabilidad
`Empleado` (abstracta)	Define el estado comun de todo empleado: nombre, tipo y salario, con su constructor, getters/setters y toString(). Declara el metodo abstracto calcularPago(), obligando a cada subclase a definir su propia formula. Al ser abstracta no puede instanciarse directamente.
`Gerente`	Subclase de Empleado. Implementa calcularPago() como salario + BONO_GERENTE (constante 1000). Unica responsabilidad: la regla de pago de un gerente.
`Desarrollador`	Subclase de Empleado. Implementa calcularPago() devolviendo el salario integro, sin bonos ni descuentos.
`Practicante`	Subclase de Empleado. Implementa calcularPago() como salario * PORCE_PRACTICANTE (constante 0.5), es decir medio salario.

2.3 com.usil.service - Capa de negocio

Clase	Responsabilidad
`EmpleadoService`	Expone calcularPagoEmpleado(Empleado). Recibe cualquier Empleado y delega en empleado.calcularPago() aprovechando el polimorfismo: el service no necesita saber si es gerente, desarrollador o practicante. Su responsabilidad es exponer la operacion de negocio, no implementar la formula.

2.4 com.usil.repository - Capa de persistencia

Clase	Responsabilidad
`EmpleadoRepository`	Expone guardarEnBaseDeDatos(String nombre). Concentra toda la logica de acceso a datos del empleado (aqui simulada con un System.out.println). Si manana cambia el motor de base de datos, este es el unico archivo que se toca.

2.5 com.usil.report - Capa de reporteria

Clase	Responsabilidad
`EmpleadoReport`	Expone generarReporte(String nombre). Concentra la generacion de reportes del empleado (simulada por consola). Si cambia el formato del reporte (PDF, Excel, HTML), el cambio queda aislado en esta clase.

3. Aplicacion del Principio de Unica Responsabilidad (SRP)

"Una clase debe tener una, y solo una, razon para cambiar." - Robert C. Martin

3.1 Punto de partida: la clase con multiples responsabilidades

En la version original, una sola clase Empleado acumulaba cuatro razones para cambiar:

```
class Empleado {
    private String nombre;
    private String tipo;
    private double salario;

    public double calcularPago() {
        if (tipo.equals("Gerente")) {
            return salario + 1000;
        } else if (tipo.equals("Desarrollador")) {
            return salario;
        } else if (tipo.equals("Practicante")) {
            return salario * 0.5;
        }
        return salario;
    }

    public void guardarEnBaseDeDatos() { ... }   // persistencia
    public void generarReporte() { ... }         // reporteria
}
```

Problemas concretos:

1. Era modelo de datos (atributos del empleado).
2. Era motor de calculo de planilla, con un if/else encadenado por tipo de empleado.
3. Era capa de acceso a datos (guardarEnBaseDeDatos).
4. Era generador de reportes (generarReporte).

Cualquier cambio en el bono de gerentes, en el motor de base de datos o en el formato del reporte obligaba a modificar y recompilar la misma clase, con riesgo de romper funcionalidad no relacionada.

3.2 Refactorizacion aplicada

a) Separacion de responsabilidades en paquetes distintos

Cada responsabilidad se extrajo a su propia clase, en su propio paquete:

Responsabilidad original	**Ahora vive en**
Estado / datos del empleado	`com.usil.model.Empleado`
Calculo del pago	Cada subclase de model + `com.usil.service.EmpleadoService`
Guardar en base de datos	`com.usil.repository.EmpleadoRepository`
Generar reporte	`com.usil.report.EmpleadoReport`

Asi, Empleado dejo de tener motivos para cambiar por razones de persistencia o de reporteria: hoy solo cambia si cambian los atributos del empleado.

b) Eliminacion del if/else por tipo mediante herencia y polimorfismo

Empleado se convirtio en clase abstracta y calcularPago() en metodo abstracto. Cada tipo de empleado implementa su propia formula:

```
public abstract class Empleado {
    public abstract double calcularPago();
}

public class Gerente extends Empleado {
    private static final double BONO_GERENTE = 1000;

    @Override
    public double calcularPago() {
        return getSalario() + BONO_GERENTE;
    }
}
```

Con esto, la regla de pago de un gerente vive solo en Gerente, y la del practicante solo en Practicante. Cada clase tiene una unica razon para cambiar: que cambie la politica salarial de ese rol. Los valores magicos (1000, 0.5) se aislaron ademas en las constantes BONO_GERENTE y PORCE_PRACTICANTE.

c) El servicio como unica puerta de entrada al calculo

```
public class EmpleadoService {

    public double calcularPagoEmpleado(Empleado empleado) {
        return empleado.calcularPago();   // polimorfismo
    }
}
```

EmpleadoService no contiene formulas: su responsabilidad es exponer la operacion de negocio. Al recibir el tipo base Empleado, Java resuelve en tiempo de ejecucion que implementacion ejecutar.

d) La clase main solo orquesta

PrincipiosSOLID se limita a construir objetos, pedir el calculo al service e imprimir. No calcula, no guarda, no reporta.

3.3 Beneficios obtenidos

- Bajo acoplamiento: un cambio en la base de datos no toca el modelo ni el reporte.
- Alta cohesion: cada clase hace una sola cosa y la hace completa.
- Extensibilidad: agregar un nuevo rol (por ejemplo Practicante Senior) solo requiere una nueva subclase de Empleado; no se modifica ninguna clase existente. Esto es, ademas, la puerta al principio Abierto/Cerrado (OCP).
- Testeabilidad: EmpleadoService, EmpleadoRepository y EmpleadoReport pueden probarse por separado.

4. Como compilar y ejecutar

```
mvn clean compile
mvn exec:java -Dexec.mainClass=com.usil.PrincipiosSOLID
```

Salida esperada:

```
Pago de Gerente:6000.0
Pago de Desarrollador:3000.0
Pago de Practicante:500.0
```

5. Observaciones sobre el estado actual del codigo

Puntos detectados al revisar el proyecto, utiles como siguiente iteracion:

5. EmpleadoRepository y EmpleadoReport no se invocan desde PrincipiosSOLID.main(). Las clases existen y cumplen su responsabilidad, pero el flujo de demostracion solo ejercita el calculo de pagos.
6. El atributo tipo quedo redundante. Con la jerarquia de subclases, el tipo de empleado ya esta representado por la propia clase (Gerente, Desarrollador, Practicante); pasarlo como String al constructor duplica esa informacion y permite estados inconsistentes, por ejemplo new Gerente("Juan", "Practicante", 5000).
7. EmpleadoService depende de la clase concreta Empleado. Para avanzar hacia el principio de Inversion de Dependencias (DIP) podria trabajarse contra una interfaz, por ejemplo Pagable.